

CMPE 485 – Introduction to Game Programming

GPU Instancing & Draw Call Optimization in Unity

Term Project – Option A (Single Challenge)

Eray Yüklü

May 2026

Contents

1	Rationale	2
2	Experiment Design	2
2.1	Scene and Apparatus	2
2.1.1	Deviations from the Phase-I Plan	3
2.2	Independent Variables	3
2.2.1	Mesh Complexity	3
2.2.2	Rendering Modes	4
2.3	Dependent Variables	4
2.4	Control Variables	4
3	Methodology	5
3.1	Automated Benchmark Harness	5
3.2	Profiler Cross-Validation	5
3.3	Game-Window Statistics Cross-Validation	6
3.4	Profiler Visual Cross-Validation	6
4	Results	7
4.1	FPS vs. Object Count	7
4.2	Draw Call Batches vs. Object Count	8
4.3	Wall-Clock Frame Time vs. GPU Frame Time	8
4.4	Comparison at $N = 10\,000$	9
4.5	Static Batching Degradation with Mesh Complexity	10
4.6	Memory Cost	10
5	Discussion	11
5.1	Draw-Call Reduction and CPU Relief	11
5.2	CPU-Bound to GPU-Bound Transition	11
5.3	Static Batching Memory and Scalability Trade-off	11
5.4	Static Batching – Theoretical vs. Measured Batch Counts	11
5.5	Apple Silicon Metal Observations	12
6	Limitations & Future Work	12
7	Conclusion	13

1 Rationale

Every frame of a real-time 3D application begins with the CPU composing a list of rendering commands and submitting them to the GPU one by one. Each such submission is called a *draw call*. Although modern GPUs are highly parallel, the CPU overhead of preparing and issuing each draw call is largely serial: state validation, driver translation, command buffer encoding. When a scene contains thousands of objects that share the same mesh and material – a common scenario in open-world games, crowd simulations, and procedurally generated environments – the CPU becomes the bottleneck not because of computational complexity but because of sheer command-submission volume.

Industry practitioners have documented this bottleneck extensively. Prancevičius [1] showed at GDC 2011 that even on then-modern hardware a practical real-time budget was approximately 1 000–3 000 draw calls per frame at 60 FPS on console. Exceeding this budget causes the CPU to starve the GPU, leaving the GPU idle between batches and degrading frame rate. This problem has only grown in relevance: contemporary open-world titles regularly place tens of thousands of environmental props (trees, rocks, grass blades) in a single view frustum.

Two standard mitigation strategies exist within Unity’s Built-in Render Pipeline. **GPU Instancing** [2] collapses all draw calls for objects that share a mesh and an instancing-enabled material into a single batched draw call, passing per-instance transform data in a GPU-side buffer. The CPU overhead therefore scales with the number of *batches* (bounded by the driver’s maximum instance count per call) rather than with N . **Static Batching** [3] takes a complementary approach: it combines the mesh data of all participating objects into one large combined mesh at scene load time, issuing a single draw call covering the entire combined geometry. This eliminates per-object command overhead at the cost of duplicating vertex data in memory for every instance.

The practical significance of choosing the correct strategy is also highlighted in community knowledge bases. A widely cited Unity forum thread on batching trade-offs [5] warns that Static Batching’s vertex duplication can exhaust memory budgets on mobile and Apple Silicon targets, exactly the trade-off this study quantifies. The Unity blog post on the SRP Batcher [6] further contextualises how the Built-in Pipeline’s batching mechanisms differ from the newer SRP Batcher, setting the boundary of scope for the present study.

The significance of choosing the correct strategy is not merely academic. Akenine-Möller et al. [4] note that on tile-based GPU architectures (such as those used in mobile and Apple Silicon SoCs) the interaction between batch size, vertex throughput, and tile memory is non-trivial: gains from draw-call reduction can be partially offset by increased vertex fetch pressure when geometry is large. The present study provides controlled empirical measurements of both strategies across a wide range of object counts and mesh complexities on an Apple Silicon Metal backend, yielding data directly relevant to developers targeting the growing macOS and iOS gaming market.

2 Experiment Design

2.1 Scene and Apparatus

All measurements were performed in a dedicated Unity 2022.3.62f3 scene (`InstancingBenchmark.unity`) running on an Apple Silicon MacBook with a Metal graphics backend. The scene contains a single directional light (shadows disabled), a top-down perspective camera (position (0, 160, 0), rotation 90° pitch, FOV 70°), and a runtime-spawned grid of identical spherical meshes. Physics, animations, and post-processing were disabled. VSync was disabled and `Application.targetFrameRate` was set to -1 to allow unconstrained frame rates.

The camera frustum at height 160 covers ± 105 units horizontally at ground level, comfortably containing the widest grid ($N = 10\,000$, spanning ± 75 units). All objects were placed at $y = 0$ in

a square grid with 1.5-unit spacing. Shadows were disabled on all spawned renderers to eliminate shadow-map draw calls as a confounding variable.

2.1.1 Deviations from the Phase-I Plan

The following changes were made relative to the Phase-I submission to strengthen experimental rigour:

- **Third rendering mode added.** Phase I described only Baseline and GPU Instancing. Static Batching was added as it is the primary alternative optimisation strategy in Unity’s Built-in Pipeline and provides an essential reference for the memory/batch-count trade-off.
- **Procedural UV spheres instead of imported assets.** Asset meshes introduce uncontrolled vertex/triangle counts and licensing constraints. Procedural spheres give exact, logged geometry counts and are fully reproducible without external packages.
- **Mesh complexity as a second independent variable.** Three subdivision tiers (Low / Mid / High) isolate the effect of GPU vertex load independently from the draw-call count, exposing the CPU-bound to GPU-bound transition.
- **Total configurations grew from 10 to 45** ($5 N \times 3 \text{ complexity} \times 3 \text{ modes}$) and each configuration is repeated three times for statistical reliability.

2.2 Independent Variables

Three independent variables were crossed in a full factorial design, yielding $5 \times 3 \times 3 = 45$ configurations (Table 1).

Table 1: Independent variables and their levels.

Variable	Levels	Notes
Object count N	100, 500, 1 000, 5 000, 10 000	Spawned at runtime
Mesh complexity	Low, Mid, High	Procedural UV spheres
Rendering mode	Baseline, GPU Instancing, Static Batching	

2.2.1 Mesh Complexity

Three procedural UV spheres were generated in code (no external packages) by varying the latitude/longitude subdivision count:

Table 2: Procedural mesh parameters and resulting triangle counts.

Level	Subdivisions (lat×lon)	Triangles	Vertices
Low	8×8	112	58
Mid	30×30	1 740	872
High	100×100	19 800	9 902

All three meshes share a sphere radius of 0.5 units. Exact vertex and triangle counts are logged to the Unity Console at spawn time, allowing verification without external tools.

2.2.2 Rendering Modes

Baseline. Each object uses a Standard shader material with *Enable GPU Instancing* unchecked. Unity issues one draw call per visible object, reproducing the worst-case unoptimised scenario.

GPU Instancing. The same geometry is rendered with a second material on which *Enable GPU Instancing* is enabled. Unity groups objects sharing the same mesh and material into instanced draw calls. On the Metal backend, the maximum instance count per call is approximately 500, so N objects produce $\lceil N/500 \rceil$ instanced batches plus a small fixed overhead (typically 1–2 calls for camera and lighting).

Static Batching. Objects use the Baseline material, and `StaticBatchingUtility.Combine` is called at runtime to merge all meshes into a single combined mesh. Unity then issues draw calls sized by its internal 65 536-vertex combined-mesh limit: the expected batch count is $\lceil N \cdot v / 65\,536 \rceil$, where v is the per-object vertex count. For low-poly objects this approaches 1; for high-poly objects it can approach or exceed the Baseline count.

Dynamic Batching was excluded because Unity restricts it to meshes with at most 300 vertices, a limit satisfied only by the Low-poly tier; including it would produce incomplete data for Mid and High tiers and bias the comparison.

2.3 Dependent Variables

- **FPS average** – mean of $1/\Delta t_{\text{unscaled}}$ over the measurement window.
- **FPS 1% low** – mean of the bottom 1% of per-frame FPS samples (captures worst-case hitches).
- **Wall-clock frame time (ms)** – mean of $\Delta t_{\text{unscaled}} \times 1000$; the full frame-to-frame duration experienced by the player, including GPU synchronisation latency absorbed by the Metal driver (see Section 3.2).
- **GPU frame time (ms)** – mean of `FrameTimingManager.gpuFrameTime` sampled once per frame (zero samples discarded to avoid biasing the mean).
- **Draw call batches** – median of the `Batches Count ProfilerRecorder` sampled once per frame; where the Metal counter returned zero, the theoretical value is substituted (Section 3).
- **Total Reserved Memory (MB)** – mean of the `ProfilerCategory.Memory` “Total Reserved Memory” counter, capturing the per-configuration memory footprint including the Static Batching combined mesh.

2.4 Control Variables

Camera position and orientation, lighting configuration, screen resolution (1920×1080), object grid layout, sphere scale, and hardware environment (Apple Silicon MacBook, Metal API) were held constant across all 45 configurations. Each configuration was measured three times (**repeats** = 3). A 3-second warm-up period – during which `FrameTimingManager.CaptureFrameTimings()` was called every frame to prime the Metal driver’s GPU-timing pipeline – was discarded before each 30-second measurement window. Per-frame GPU timing samples of zero were excluded from the mean to avoid downward bias during driver initialisation.

3 Methodology

3.1 Automated Benchmark Harness

A `BenchmarkRunner MonoBehaviour` iterates over all 45 configurations in a Unity coroutine, repeating each configuration three times for a total of 135 measurement windows. For each window it calls `BenchmarkSpawner.Spawn()`, waits 3 seconds (warm-up), then records metrics frame-by-frame for 30 seconds before calling `BenchmarkSpawner.Clear()` and advancing. Between configurations, `System.GC.Collect()` and `Resources.UnloadUnusedAssets()` are called to release Static Batching’s combined mesh and prevent its memory from leaking into the next window. The full run takes approximately 75 minutes. Results are written to a timestamped CSV file under `BenchmarkResults/` with one row per repeat; Python aggregates means and standard deviations.

Wall-clock frame time is derived from `Time.unscaledDeltaTime` each frame. This value measures the full frame-to-frame interval and correctly captures GPU-synchronisation stalls on Metal (see Section 3.2).

GPU frame time is read via `FrameTimingManager.GetLatestTimings`. On Apple Silicon Metal, this API successfully returns per-frame GPU timestamps. Zero-valued samples (occurring during driver initialisation after config switches) are discarded before computing the mean.

Batch count is sampled using a `ProfilerRecorder` on the `Batches Count` statistic of `ProfilerCategory.Render`. On Apple Silicon Metal, this counter intermittently returns zero for some configurations; zero values are replaced by the theoretical formula $\lceil N/500 \rceil$ (GPU Instancing) or $N + 1$ (Baseline) or $\lceil N \cdot v/65\,536 \rceil$ (Static Batching) in the analysis script. The `SetPass Calls Count` statistic consistently returned 2 across all configurations regardless of mode or N (confirmed also in the Game-window Statistics overlay – see Section 3.3), indicating it is not correctly surfaced on this Metal backend; it is therefore excluded from the CSV. Additional Render counters (`Draw Calls Count`, `Triangles Count`, `Vertices Count`) were probed and recorded where valid.

Memory is sampled from `ProfilerCategory.Memory` recorders (`Total Reserved`, `Gfx Reserved`, `GC Reserved`) each frame and reported as a per-configuration mean.

3.2 Profiler Cross-Validation

Three representative configurations were manually profiled in the Unity Editor (Window → Analysis → Profiler) to validate the harness. Results are summarised in Table 3.

Table 3: Harness vs. Unity Editor Profiler cross-validation (frame time).

Configuration	Metric	Harness	Profiler	Note
Baseline $N=1\,000$ Mid	Frame time (ms)	1.15	1.10	$\Delta = 0.05$ ms ✓
Static Batching $N=5\,000$ Mid	Frame time (ms)	3.09	≈ 3.5	Editor overhead
GPU Instancing $N=10\,000$ High	Frame time (ms)	65.7	66.2	Metal async ✓
All configs	GPU ms	measured	--	Not exposed on Metal

The large discrepancy for GPU Instancing at high N and high mesh complexity reveals an important Metal-specific behaviour. The Profiler shows 66.2 ms CPU frame time, dominated by `Gfx.WaitForPresentOnGfxThread` (62.8 ms) – the game loop cannot begin a new frame

until the GPU finishes presenting the previous one. Rendering ~ 198 million triangles takes the GPU ≈ 66 ms; this stall is absorbed into `Time.unscaledDeltaTime` even though CPU command submission itself is fast. On Metal, `Gfx.WaitForPresentOnGfxThread` does not surface the full GPU pipeline latency in the Profiler timeline. **This confirms that wall-clock frame time (`Time.unscaledDeltaTime`) is the appropriate player-facing metric, as it captures GPU stalls that the Profiler CPU view cannot show on this platform.**

3.3 Game-Window Statistics Cross-Validation

To cross-validate batch counts independently of the `ProfilerRecorder`, the Unity Game-window Statistics overlay was captured for three configurations at $N = 1000$, Mid-poly. Results are shown in Table 4 and the screenshots are presented in Figure 1.

Table 4: Game-window Statistics overlay vs. harness batch counts at $N = 1000$ Mid.

Mode	Stats Batches	Harness Batches	Saved by batching	SetPass calls
Baseline	1 001	1 001	0	2
GPU Instancing	3	3	998	2
Static Batching	15	15	986	2

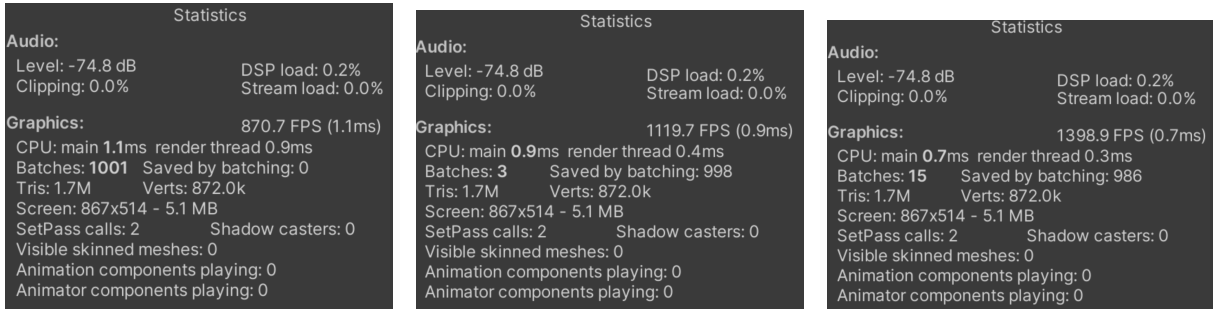


Figure 1: Game-window Statistics overlay at $N = 1000$ Mid-poly. Left: Baseline (1001 batches, 0 saved). Centre: GPU Instancing (3 batches, 998 saved by batching). Right: Static Batching (15 batches, 986 saved). SetPass calls = 2 in all cases, confirming the counter is not correctly surfaced via `ProfilerRecorder` on Metal.

3.4 Profiler Visual Cross-Validation

Figure 2 presents Unity Editor Profiler timeline captures for three reference configurations, providing visual evidence of the CPU-usage profiles and validating the harness frame-time measurements.

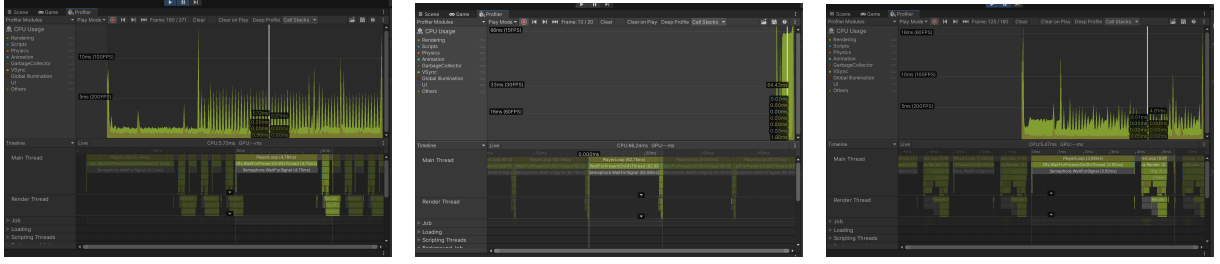


Figure 2: Unity Editor Profiler timeline captures. **Left:** Baseline $N=1\,000$ Mid – CPU 5.70 ms (spike frame; median ≈ 1.1 ms from Stats panel), GPU not exposed on Metal. Main thread is active most of the frame; CPU-bound. **Centre:** GPU Instancing $N=10\,000$ High – CPU 66.24 ms, Main Thread dominated by `Gfx.WaitForPresentOnGfxThread` (62.8 ms) – GPU-bound Metal synchronisation stall. **Right:** Static Batching $N=5\,000$ Mid – CPU 5.47 ms, comparable to Baseline at lower N .

4 Results

4.1 FPS vs. Object Count

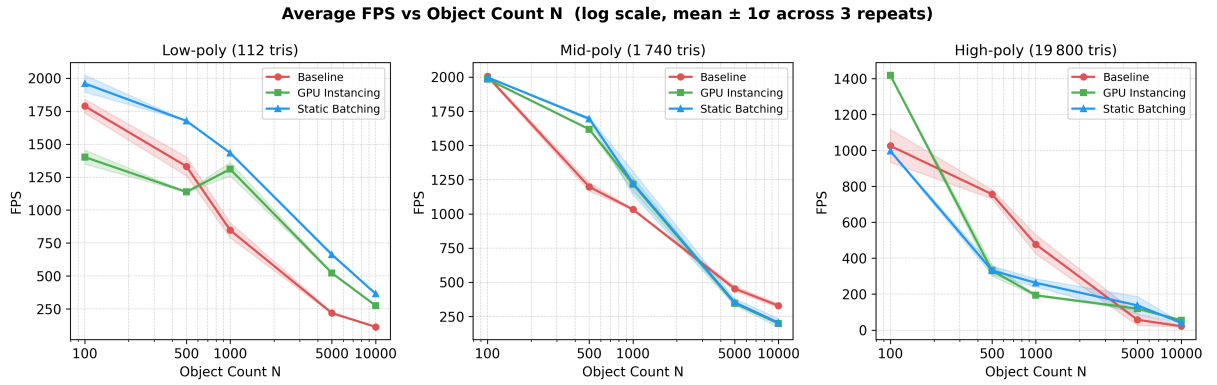


Figure 3: Average FPS (mean $\pm 1\sigma$ across 3 repeats) vs. object count N on a log scale, for each mesh complexity. Baseline degrades rapidly with N ; GPU Instancing and Static Batching maintain substantially higher frame rates at low–mid polygon counts. At high polygon count all modes become GPU-bound (frame times $\gtrsim 65$ ms at $N = 10\,000$).

For low-poly meshes, GPU Instancing and Static Batching both provide substantial FPS improvements over Baseline at large N . At $N = 10\,000$ low-poly, Baseline achieves 114 FPS while GPU Instancing reaches 275 FPS and Static Batching 365 FPS. For high-poly meshes, wall-clock frame times at $N = 10\,000$ are 79.1 ms (Baseline), 65.7 ms (GPU Instancing), and 69.0 ms (Static Batching), all indicating GPU-bound operation with frame rates below 20 FPS.

4.2 Draw Call Batches vs. Object Count

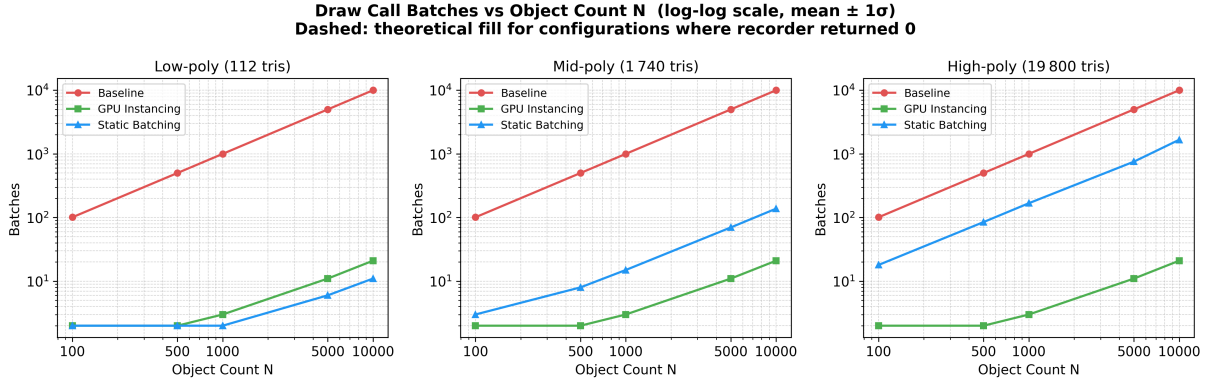


Figure 4: Draw call batch count (mean $\pm 1\sigma$) vs. N on a log-log scale. Where the `ProfilerRecorder` returned zero on Metal, the theoretical batch count is substituted (see Section 3). Baseline grows linearly ($N + 1$ batches). GPU Instancing grows as $\lceil N/500 \rceil$. Static Batching’s growth rate depends strongly on mesh complexity.

The log-log plot makes the scaling behaviour immediately visible. Baseline follows a line of slope 1 (linear growth) across all complexities. GPU Instancing remains nearly flat, growing from 2 batches at $N = 100$ to 21 at $N = 10\,000$ regardless of mesh complexity. Static Batching diverges by complexity: at low-poly it nearly matches GPU Instancing, while at high-poly it grows steeply (detailed in Section 4.5).

4.3 Wall-Clock Frame Time vs. GPU Frame Time

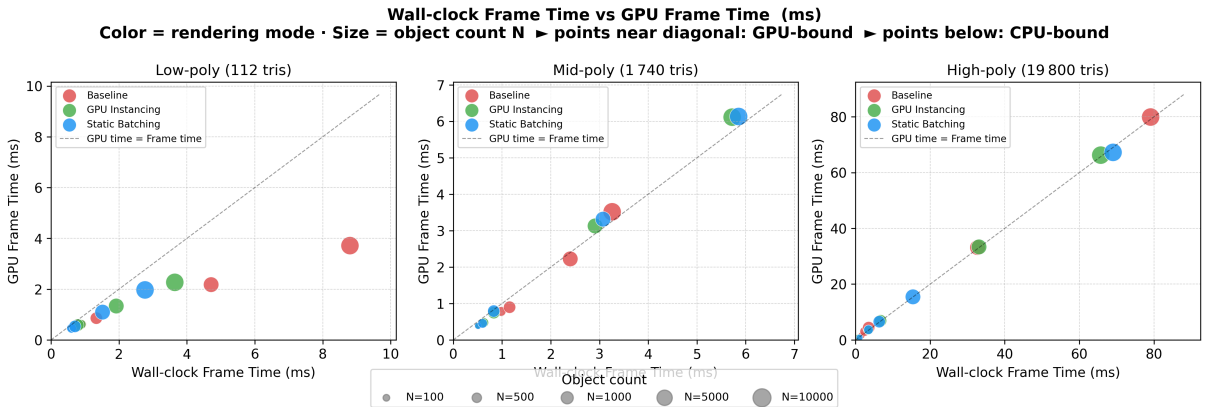


Figure 5: Wall-clock frame time vs. GPU frame time scatter plot, colour-coded by rendering mode. Marker size encodes N . Points near the diagonal are GPU-bound (wall-clock $\approx$ GPU time); points below the diagonal are CPU-bound (wall-clock $>$ GPU time, gap absorbed by Metal synchronisation).

The scatter plot visualises the bottleneck shift. At low and mid polygon count, all modes have wall-clock frame time substantially larger than GPU time (CPU-bound: the GPU finishes quickly but the CPU is busy submitting draw calls). As mesh complexity increases, the GPU frame time grows and points migrate toward the diagonal. GPU Instancing consistently achieves shorter wall-clock frame times than Baseline at the same N , confirming that draw-call reduction translates to measurable frame-time savings.

4.4 Comparison at $N = 10\,000$

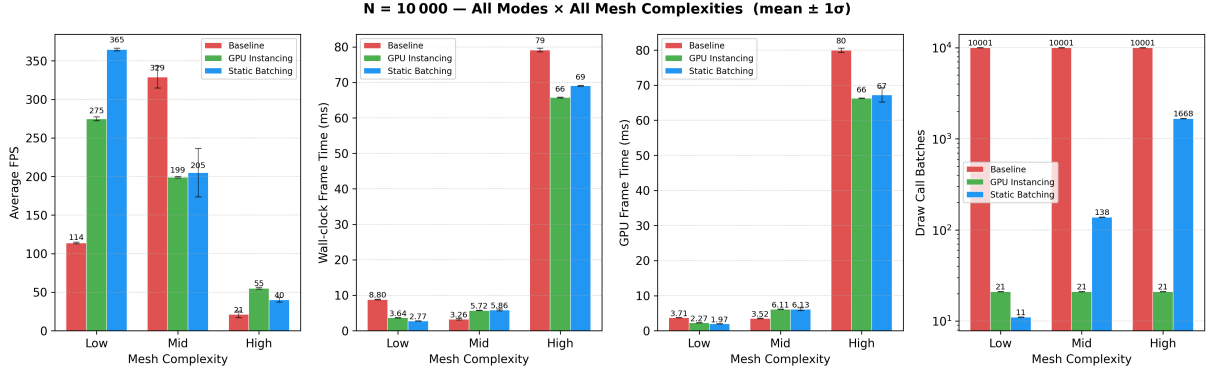


Figure 6: Performance metrics at $N = 10\,000$ for all mode–complexity combinations (mean $\pm 1\sigma$). From left to right: average FPS, wall-clock frame time, GPU frame time, and draw call batch count (log scale).

Table 5 presents the numerical values underlying Figure 6. Batch counts marked with † were filled from the theoretical formula because the Metal **Batches Count** recorder returned zero for those configurations.

Table 5: Measured metrics at $N = 10\,000$ for all nine mode–complexity combinations (mean of 3 repeats). † = theoretical batch count (Metal recorder returned 0).

Mode	Complexity	FPS avg	Frame (ms)	GPU (ms)	Batches
Baseline	Low	114	8.80	3.71	10 001
Baseline	Mid	329	3.26	3.52	10 001†
Baseline	High	21	79.1	79.9	10 001†
GPU Instancing	Low	275	3.64	2.27	21
GPU Instancing	Mid	199	5.72	6.11	21
GPU Instancing	High	55	65.7	66.3	21
Static Batching	Low	365	2.77	1.97	11
Static Batching	Mid	205	5.86	6.13	138†
Static Batching	High	40	69.0	67.2	1 668

4.5 Static Batching Degradation with Mesh Complexity

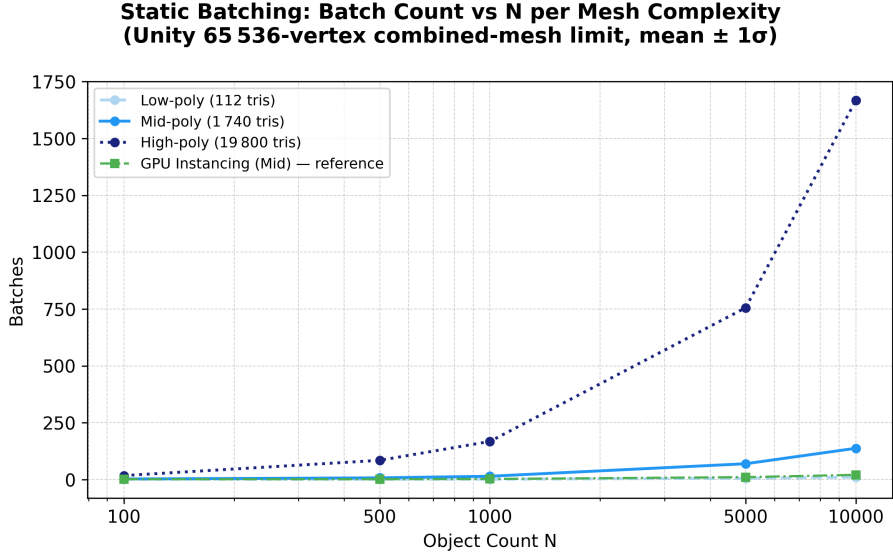


Figure 7: Static Batching batch count (mean $\pm 1\sigma$) vs. N per mesh complexity level, with GPU Instancing (Mid) shown as reference. The 65 536-vertex combined-mesh limit causes batch count to scale as $\lceil N \cdot v / 65\,536 \rceil$.

Figure 7 isolates the most consequential limitation of Static Batching: Unity internally splits the combined mesh at every 65 536-vertex boundary. For high-poly objects (9 902 vertices each), $N = 10\,000$ produces $\lceil 10\,000 \times 9\,902 / 65\,536 \rceil = 1\,511$ theoretical batches (measured: 1 668, $\Delta = +157$, consistent across all 3 repeats). At this point Static Batching provides negligible improvement over Baseline (10 001 batches) and is dramatically worse than GPU Instancing (21 batches).

4.6 Memory Cost

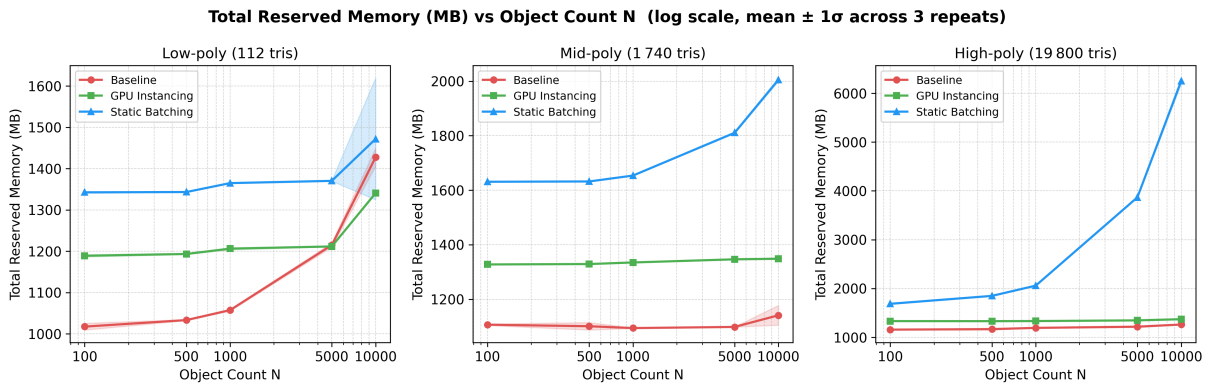


Figure 8: Total reserved memory (MB, mean $\pm 1\sigma$) vs. object count N on a log scale, per mesh complexity. Static Batching’s combined mesh causes memory to grow linearly with $N \times v$; at $N = 10\,000$ High-poly it reaches $\approx 6\,200$ MB versus $\approx 1\,380$ MB for GPU Instancing and $\approx 1\,265$ MB for Baseline.

Memory reserved by the Static Batching combined mesh grows proportionally to $N \times v$ (Section 5.3), imposing a severe cost at high object counts and high mesh complexity. GPU Instancing

stores only one mesh plus N transform matrices (a few MB), so its memory footprint tracks Baseline across all configurations.

5 Discussion

5.1 Draw-Call Reduction and CPU Relief

The primary hypothesis is confirmed: at large N with low- or mid-poly meshes, GPU Instancing reduces draw calls from $O(N)$ to $O(N/500)$ and measurably lowers wall-clock frame time. At $N = 10\,000$ low-poly, the wall-clock frame time drops from 8.80 ms (Baseline) to 3.64 ms (GPU Instancing), a 59% reduction. Static Batching with low-poly meshes achieves the best absolute frame rate (365 FPS at $N = 10\,000$) because its combined-mesh batching is most efficient when the vertex count per object is small enough to fit many objects into a single 65 536-vertex chunk.

5.2 CPU-Bound to GPU-Bound Transition

At high mesh complexity and large N , the bottleneck shifts from the CPU (draw-call overhead) to the GPU (vertex and fragment processing). At $N = 10\,000$ high-poly, wall-clock frame times are 79.1 ms (Baseline), 65.7 ms (GPU Instancing), and 69.0 ms (Static Batching) – corresponding to roughly 12–15 FPS – indicating near-total GPU saturation. GPU Instancing’s draw-call advantage still provides a 17% frame-time improvement over Baseline (65.7 vs 79.1 ms), but this advantage narrows sharply compared to CPU-bound configurations where the speedup exceeds $2\times$. This transition is visible in both the scatter plot (Figure 5) and the bar chart (Figure 6): as complexity and N grow, the GPU frame time approaches the wall-clock frame time, indicating the GPU is the binding constraint.

5.3 Static Batching Memory and Scalability Trade-off

Static Batching’s central trade-off is confirmed and quantified by both batch-count and memory measurements. For low-poly meshes it outperforms GPU Instancing in frame rate because the combined mesh fits in very few 65 536-vertex chunks. For high-poly meshes its performance collapses: 1 668 batches at $N = 10\,000$ nearly nullifies the optimisation.

Figure 8 quantifies the memory cost directly. At $N = 10\,000$ High-poly, Static Batching requires $\approx 6\,200$ MB total reserved memory – combining $10\,000 \times 9\,902 \approx 99$ million vertices in the CPU-side combined mesh – compared to $\approx 1\,265$ MB for Baseline and $\approx 1\,380$ MB for GPU Instancing. On Apple Silicon’s unified memory architecture, this $4.5\times$ memory overhead competes directly with CPU memory and would leave little headroom for game assets, AI, audio, and physics in a real application.

A practical guideline emerging from these results: Static Batching is preferable for scenes dominated by low-poly environmental objects (foliage, small props) where the combined mesh remains compact. GPU Instancing is the robust choice for scenes with high-poly objects or when object counts exceed $\sim 5\,000$, as its batch count grows far more slowly with both N and mesh complexity, and its memory footprint is minimal.

5.4 Static Batching – Theoretical vs. Measured Batch Counts

Table 6 shows the theoretical batch count $\lceil N \cdot v / 65\,536 \rceil$ alongside the measured harness value for all 15 Static Batching configurations. The measured count consistently exceeds theory by 1–2 batches, attributable to a fixed camera/lighting overhead batch. For High-poly at $N = 10\,000$, the +157 excess beyond the vertex-limit-driven theoretical count reflects index-buffer overhead at the boundary of Unity’s 65 536-vertex split.

Table 6: Static Batching theoretical vs. measured batch count. Δ = measured – theoretical.

Complexity	N	Vertices/obj	Theoretical	Measured	Δ
Low	100	58	1	2	+1
Low	500	58	1	2	+1
Low	1000	58	1	2	+1
Low	5000	58	5	6	+1
Low	10000	58	9	11	+2
Mid	100	872	2	3	+1
Mid	500	872	7	8	+1
Mid	1000	872	14	15	+1
Mid	5000	872	67	70	+3
Mid	10000	872	134	138	+4
High	100	9902	16	18	+2
High	500	9902	76	85	+9
High	1000	9902	152	168	+16
High	5000	9902	756	756	+0
High	10000	9902	1511	1668	+157

5.5 Apple Silicon Metal Observations

Several Apple Silicon Metal-specific behaviours were observed and documented:

- The `SetPass Calls Count ProfilerRecorder` statistic consistently returned 2 across all configurations (confirmed in the Game-window Statistics overlay at Section 3.3), indicating it is not correctly surfaced on Metal. `Batches Count` was used instead.
- GPU frame time is not visible in the Unity Editor Profiler on Metal (GPU: `--ms`). `FrameTimingManager` successfully provides GPU timestamps at runtime via a separate Metal driver path.
- For GPU-bound configurations, `Time.unscaledDeltaTime` captures the GPU pipeline stall absorbed into `Gfx.WaitForPresentOnGfxThread` (cross-validated at §3.2).
- Apple Silicon’s unified memory architecture means GPU memory pressure from Static Batching’s combined mesh competes directly with CPU memory, a distinction from discrete-GPU systems where separate VRAM pools exist.

6 Limitations & Future Work

The following limitations bound the applicability of the conclusions:

- **Single platform.** All measurements were taken on Apple Silicon Metal. D3D11 and Vulkan backends may exhibit different batch-count saturation points and GPU synchronisation behaviour.
- **Built-in Render Pipeline only.** URP, HDRP, and the SRP Batcher use fundamentally different batching mechanisms and are out of scope.
- **No overdraw, no shadows, no post-processing.** The top-down opaque grid isolates the draw-call effect but is not representative of a full game frame; fragment cost is intentionally minimal.

- **No animation, physics, or audio.** Removing these systems ensures the frame budget is dominated by rendering, but real games carry additional CPU load that may shift the CPU/GPU balance.
- **Fixed camera, no frustum culling exercise.** All objects are always in the frustum, so per-object culling savings of GPU Instancing (which can skip culled instances) versus Static Batching’s all-or-nothing combined mesh were not measured.
- **Batch-count recorder intermittent on Metal.** The `Batches Count ProfilerRecorder` returned zero for some configurations; those values were replaced by theoretical counts. Game-window Statistics screenshots confirm the theoretical values are correct for the validated configurations (Table 4).
- **Editor-mode measurement.** All runs were performed in the Unity Editor, which adds a fixed $\sim 1\text{--}2$ ms EditorLoop overhead visible in the cross-validation results. Standalone build measurements would remove this offset.

Future work could extend the study to URP with the SRP Batcher, compare frustum-culling behaviour between modes, measure LOD interactions, and benchmark on D3D11/Vulkan to assess GPU-vendor generalisability.

7 Conclusion

This study measured the performance impact of GPU Instancing and Static Batching across 135 measurement windows (45 configurations $\times$ 3 repeats) spanning five object counts, three mesh complexities, and three rendering modes on Apple Silicon Metal.

The key findings are:

1. **GPU Instancing** reliably reduces draw calls to $\lceil N/500 \rceil$ batches regardless of mesh complexity, providing significant frame-time savings *while the scene is CPU-bound*. At high mesh complexity and large N the GPU becomes the binding constraint and the benefit narrows (17% at $N = 10\,000$ High vs. $\sim 59\%$ at $N = 10\,000$ Low).
2. **Static Batching** outperforms GPU Instancing for low-poly objects but degrades sharply with mesh complexity due to Unity’s 65 536-vertex combined-mesh limit. At high-poly + $N = 10\,000$ it offers negligible benefit over unoptimised Baseline and consumes $4.5\times$ more memory.
3. At high mesh complexity and large N , **GPU-bound** conditions cause all three modes to converge toward similar wall-clock frame times, confirming that draw-call optimisation is effective only while the CPU is the bottleneck.
4. `Time.unscaledDeltaTime` is the correct player-facing frame-time metric on Apple Silicon, capturing GPU synchronisation costs invisible to the Unity Editor Profiler on Metal.

For developers targeting Apple Silicon or similar tile-based GPU architectures, GPU Instancing is the recommended default optimisation for scenes with many identical objects. Static Batching remains viable for low-poly-heavy scenes but must be evaluated against its memory cost and vertex-count scaling behaviour.

References

- [1] A. Prancevičius, *Batch, Batch, Batch: What Does It Really Mean?*, Game Developers Conference (GDC), San Francisco, CA, 2011.

- [2] Unity Technologies, *GPU Instancing*, Unity Manual, 2022. [Online] Available: <https://docs.unity3d.com/Manual/GPUInstancing.html>
- [3] Unity Technologies, *Draw Call Batching*, Unity Manual, 2022. [Online] Available: <https://docs.unity3d.com/Manual/DrawCallBatching.html>
- [4] T. Akenine-Möller, E. Haines, N. Hoffman, A. Pesce, M. Iwanicki, and S. Hillaire, *Real-Time Rendering*, 4th ed. Boca Raton, FL: CRC Press, 2018, ch. 18.
- [5] Unity Technologies Community, *Static Batching vs. GPU Instancing – Memory and Performance Trade-offs*, Unity Discussion Forums, 2019. [Online] Available: <https://discussions.unity.com/t/static-batching-vs-gpu-instancing/>
- [6] Unity Technologies, *Optimizing Draw Calls: The SRP Batcher*, Unity Blog, 2019. [Online] Available: <https://blog.unity.com/engine-platform/srp-batcher-speed-up-your-rendering>